TBD

TBD

Azeglio Martinelli

2026-03-19

Table of contents

1 Summary **1**

2 Requirements & Configuration **1**

3 ELT Process **2**
3.1 DB Setup 2
3.2 Raw Data Loading 3
3.3 Data Inspection 4
3.4 Data Cleaning & Transformation 8

4 Class Diagramm **12**

5 Analysis **13**
5.1 Launch success rate by rocket 13
5.2 Most active launchpads 14
5.3 Launch frequency over time 15
5.4 Core reuse analysis 16

1 Summary

2 Requirements & Configuration

```
import sys
!{sys.executable} -m pip install -r requirements.txt

import pymongo
from pymongo.errors import ConnectionFailure
import pprint as pp
import pandas as pd
```

```python
import requests
import json
import time
import string
import os
from dotenv import load_dotenv
```

```python
# API and Database details
load_dotenv()

# MongoDB
CNX_STR = os.getenv("MONGO_URI")
DB_NAME = "spacexdb"

# SpaceX API endpoints
LAUNCHES_URL = "https://api.spacexdata.com/v4/launches"
ROCKETS_URL = "https://api.spacexdata.com/v4/rockets"
LAUNCHPADS_URL = "https://api.spacexdata.com/v4/launchpads"
```

3 ELT Process

3.1 DB Setup

```python
# Connection to MongoDB
client = pymongo.MongoClient(CNX_STR)
db = client[DB_NAME]

launches_raw = db["launches_raw"]
rockets_raw = db["rockets_raw"]
launchpads_raw = db["launchpads_raw"]
```

```python
# Test connection
try:
    # The ping command is cheap and does not require auth.
    client.admin.command('ping')
    print('Ping succesful!')
    print(f"Available collections: : {(db.list_collection_names())}")
except ConnectionFailure:
    print("Server not available!")
```

```
Ping succesful!
Available collections: : ['rockets_stage', 'launchpads_raw', 'rockets_raw', 'launchpads_stage', 'laun
```

```
# OPTIONAL: delete raw data
launches_raw.drop()
rockets_raw.drop()
launchpads_raw.drop()

print("launches_raw:", launches_raw.count_documents({}))
print("rockets_raw:", rockets_raw.count_documents({}))
print("launchpads_raw:", launchpads_raw.count_documents({}))
```

3.2 Raw Data Loading

```
# Load raw launches data into launches_raw
response = requests.get(LAUNCHES_URL)
response.raise_for_status()

launches_data = response.json()

print(f"Launches downloaded: {len(launches_data)}")

if launches_data:
    launches_raw.insert_many(launches_data)

print("Documents in launches_raw:", launches_raw.count_documents({}))

# Load raw rockets data into rockets_raw
response = requests.get(ROCKETS_URL)
response.raise_for_status()

rockets_data = response.json()

print(f"Rockets downloaded: {len(rockets_data)}")

if rockets_data:
    rockets_raw.insert_many(rockets_data)

print("Documents in rockets_raw:", rockets_raw.count_documents({}))

# Load raw launchpads data into launchpads_raw
response = requests.get(LAUNCHPADS_URL)
response.raise_for_status()

launchpads_data = response.json()

print(f"Launch pads downloaded: {len(launchpads_data)}")

if launchpads_data:
```

```
    launchpads_raw.insert_many(launchpads_data)

print("Documents in launchpads_raw:", launchpads_raw.count_documents({}))
```

```
Launches downloaded: 205
Documents in launches_raw: 820
Rockets downloaded: 4
Documents in rockets_raw: 16
Launch pads downloaded: 6
Documents in launchpads_raw: 24
```

3.3 Data Inspection

```
# Quick sanity check
print("launches_raw sample:")
print(launches_raw.find_one({}, {"name": 1, "date_utc": 1, "rocket": 1, "launchpad": 1}))

print("\nrockets_raw sample:")
print(rockets_raw.find_one({}, {"name": 1, "cost_per_launch": 1, "success_rate_pct": 1}))

print("\nlaunchpads_raw sample:")
print(launchpads_raw.find_one({}, {"name": 1, "full_name": 1, "locality": 1, "region": 1}))
```

```
launches_raw sample:
{'_id': ObjectId('69bbe447a81f9c50230cb26b'), 'rocket': '5e9d0d95eda69955f709d1eb', 'launchpad': '5e9
03-24T22:30:00.000Z'}

rockets_raw sample:
{'_id': ObjectId('69bbe449a81f9c50230cb338'), 'name': 'Falcon 1', 'cost_per_launch': 6700000, 'succes

launchpads_raw sample:
{'_id': ObjectId('69bbe449a81f9c50230cb33c'), 'name': 'VAFB SLC 3W', 'full_name': 'Vandenberg Space F
```

```
# check for null/missing fields
important_fields = ["id", "name", "date_utc", "success", "rocket", "launchpad", "cores", "payloads"]

for field in important_fields:
    missing_or_null = launches_raw.count_documents({
        "$or": [
            {field: {"$exists": False}},
            {field: None}
        ]
    })
    print(f"{field}: {missing_or_null}")
```

```
id: 0
name: 0
date_utc: 0
success: 76
rocket: 0
launchpad: 0
cores: 0
payloads: 0
```

```python
# check date quality
pp.pprint(list(launches_raw.aggregate([
    {
        "$project": {
            "name": 1,
            "date_utc": 1,
            "date_type": {"$type": "$date_utc"}
        }
    },
    {"$limit": 10}
])))
```

```
[{'_id': ObjectId('69bbe447a81f9c50230cb26b'),
  'date_type': 'string',
  'date_utc': '2006-03-24T22:30:00.000Z',
  'name': 'FalconSat'},
 {'_id': ObjectId('69bbe447a81f9c50230cb26c'),
  'date_type': 'string',
  'date_utc': '2007-03-21T01:10:00.000Z',
  'name': 'DemoSat'},
 {'_id': ObjectId('69bbe447a81f9c50230cb26d'),
  'date_type': 'string',
  'date_utc': '2008-08-03T03:34:00.000Z',
  'name': 'Trailblazer'},
 {'_id': ObjectId('69bbe447a81f9c50230cb26e'),
  'date_type': 'string',
  'date_utc': '2008-09-28T23:15:00.000Z',
  'name': 'RatSat'},
 {'_id': ObjectId('69bbe447a81f9c50230cb26f'),
  'date_type': 'string',
  'date_utc': '2009-07-13T03:35:00.000Z',
  'name': 'RazakSat'},
 {'_id': ObjectId('69bbe447a81f9c50230cb270'),
  'date_type': 'string',
  'date_utc': '2010-06-04T18:45:00.000Z',
  'name': 'Falcon 9 Test Flight'},
 {'_id': ObjectId('69bbe447a81f9c50230cb271'),
  'date_type': 'string',
  'date_utc': '2010-12-08T15:43:00.000Z',
```

```
  'name': 'COTS 1'},
 {'_id': ObjectId('69bbe447a81f9c50230cb272'),
  'date_type': 'string',
  'date_utc': '2012-05-22T07:44:00.000Z',
  'name': 'COTS 2'},
 {'_id': ObjectId('69bbe447a81f9c50230cb273'),
  'date_type': 'string',
  'date_utc': '2012-10-08T00:35:00.000Z',
  'name': 'CRS-1'},
 {'_id': ObjectId('69bbe447a81f9c50230cb274'),
  'date_type': 'string',
  'date_utc': '2013-03-01T19:10:00.000Z',
  'name': 'CRS-2'}]
```

```
# check success values
pp.pprint(list(launches_raw.aggregate([
    {
        "$group": {
            "_id": "$success",
            "count": {"$sum": 1}
        }
    },
    {"$sort": {"count": -1}}
])))
```

```
[{'_id': True, 'count': 724},
 {'_id': None, 'count': 76},
 {'_id': False, 'count': 20}]
```

```
# inspect array sizes of cores and payloads
pp.pprint(list(launches_raw.aggregate([
    {
        "$project": {
            "name": 1,
            "core_count": {"$size": {"$ifNull": ["$cores", []]}},
            "payload_count": {"$size": {"$ifNull": ["$payloads", []]}}
        }
    },
    {"$limit": 10}
])))
```

```
[{'_id': ObjectId('69bbe447a81f9c50230cb26b'),
  'core_count': 1,
  'name': 'FalconSat',
  'payload_count': 1},
 {'_id': ObjectId('69bbe447a81f9c50230cb26c'),
```

```
  'core_count': 1,
  'name': 'DemoSat',
  'payload_count': 1},
 {'_id': ObjectId('69bbe447a81f9c50230cb26d'),
  'core_count': 1,
  'name': 'Trailblazer',
  'payload_count': 2},
 {'_id': ObjectId('69bbe447a81f9c50230cb26e'),
  'core_count': 1,
  'name': 'RatSat',
  'payload_count': 1},
 {'_id': ObjectId('69bbe447a81f9c50230cb26f'),
  'core_count': 1,
  'name': 'RazakSat',
  'payload_count': 1},
 {'_id': ObjectId('69bbe447a81f9c50230cb270'),
  'core_count': 1,
  'name': 'Falcon 9 Test Flight',
  'payload_count': 1},
 {'_id': ObjectId('69bbe447a81f9c50230cb271'),
  'core_count': 1,
  'name': 'COTS 1',
  'payload_count': 2},
 {'_id': ObjectId('69bbe447a81f9c50230cb272'),
  'core_count': 1,
  'name': 'COTS 2',
  'payload_count': 1},
 {'_id': ObjectId('69bbe447a81f9c50230cb273'),
  'core_count': 1,
  'name': 'CRS-1',
  'payload_count': 2},
 {'_id': ObjectId('69bbe447a81f9c50230cb274'),
  'core_count': 1,
  'name': 'CRS-2',
  'payload_count': 1}]
```

```python
# check launches empty cores and payloads
print("Launches with empty/missing cores:",
      launches_raw.count_documents({
          "$expr": {"$eq": [{"$size": {"$ifNull": ["$cores", []]}}, 0]}
      }))

print("Launches with empty/missing payloads:",
      launches_raw.count_documents({
          "$expr": {"$eq": [{"$size": {"$ifNull": ["$payloads", []]}}, 0]}
      }))
```

```
Launches with empty/missing cores: 0
```

```
Launches with empty/missing payloads: 48
```

3.4 Data Cleaning & Transformation

```
# OPTIONAL: delete stage data
launches_stage.drop()
launchpads_stage.drop()
rockets_stage.drop()

print("launches_stage:", launches_stage.count_documents({}))
print("rockets_stage:", rockets_stage.count_documents({}))
print("launchpads_stage:", launchpads_stage.count_documents({}))
```

```
rockets_stage = db["rockets_stage"]

pipeline = [
    {
        "$set": {
            "_id": "$id",
            "first_flight_date": {"$dateFromString": {"dateString": "$first_flight"}}

        }
    },
    {
        "$project": {
            "_id": 1,
            "name": 1,
            "cost_per_launch": 1,
            "success_rate_pct": 1,
            "country": 1,
            "company": 1,
            "first_flight_date": 1
        }
    },
    {
        "$merge": "rockets_stage"
    }
]

list(rockets_raw.aggregate(pipeline))
print(rockets_stage.count_documents({}))
```

```
4
```

```
launchpads_stage = db["launchpads_stage"]

pipeline = [
    {
        "$set": {
            "_id": "$id",
            "location": {"$concat": ["$locality", ", ", "$region"]}
        }
    },
    {
        "$project": {
            "_id": 1,
            "name": 1,
            "full_name": 1,
            "locality": 1,
            "region": 1,
            "latitude": 1,
            "longitude": 1,
            "launch_attempts": 1,
            "launch_successes": 1,
            "location": 1
        }
    },
    {
        "$merge": "launchpads_stage"
    }
]

list(launchpads_raw.aggregate(pipeline))
print(launchpads_stage.count_documents({}))
```

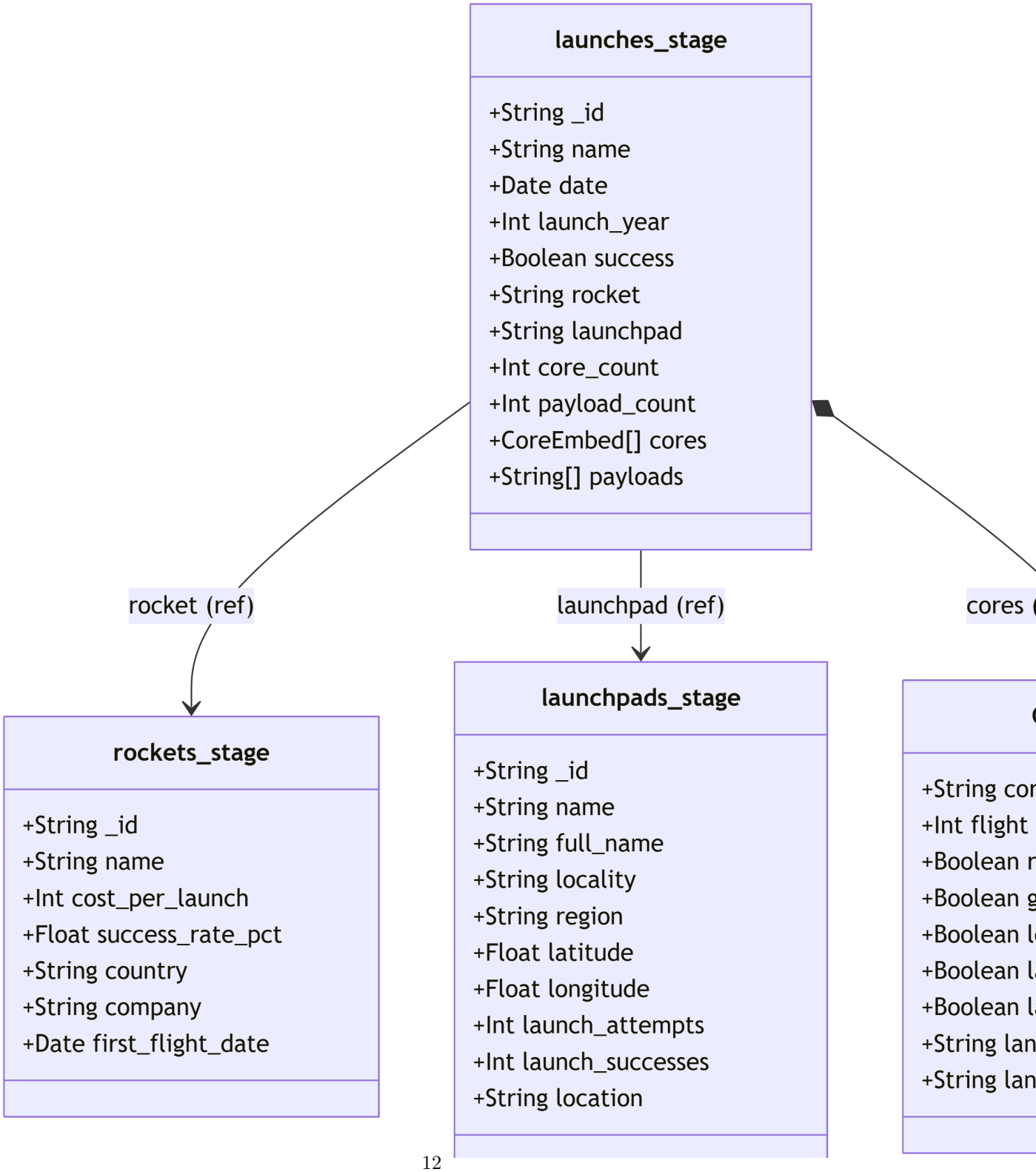
6

```
launches_stage = db["launches_stage"]

pipeline = [
    {
        "$set": {
            "_id": "$id",
            "date": {"$dateFromString": {"dateString": "$date_utc"}}
        }
    },
    {
        "$set": {
            "launch_year": {"$year": "$date"},
            "core_count": {"$size": {"$ifNull": ["$cores", []]}},
            "payload_count": {"$size": {"$ifNull": ["$payloads", []]}}
```

```
        }
    },
    {
        "$project": {
            "_id": 1,
            "name": 1,
            "date": 1,
            "launch_year": 1,
            "success": 1,
            "rocket": 1,
            "launchpad": 1,
            "cores": 1,
            "payloads": 1,
            "core_count": 1,
            "payload_count": 1
        }
    },
    {
        "$merge": "launches_stage"
    }
]

list(launches_raw.aggregate(pipeline))
print(launches_stage.count_documents({}))
```

```
205
```

4 Class Diagramm



5 Analysis

5.1 Launch success rate by rocket

```
# exclude upcoming launches without a result
match = {"$match": {"success": {"$in": [True, False]}}}

# join rocket details from rockets_stage by rocket id
lookup = {"$lookup": {
    "from": "rockets_stage",
    "localField": "rocket",
    "foreignField": "_id",
    "as": "rocket_info"
}}

# flatten rocket_info array into a single object
unwind = {"$unwind": "$rocket_info"}

# group by rocket name and count launches, successes, failures
group = {"$group": {
    "_id": "$rocket_info.name",
    "total_launches": {"$sum": 1},
    "successful": {"$sum": {"$cond": ["$success", 1, 0]}},
    "failed": {"$sum": {"$cond": ["$success", 0, 1]}},
    "cost_per_launch": {"$first": "$rocket_info.cost_per_launch"}
}}

# calculate success rate and clean up output fields
project = {"$project": {
    "_id": 0,
    "rocket": "$_id",
    "total_launches": 1,
    "successful": 1,
    "failed": 1,
    "success_rate_pct": {"$round": [
        {"$multiply": [{"$divide": ["$successful", "$total_launches"]}, 100]},
        1
    ]},
    "cost_per_launch": 1
}}

# sort by most launched rocket first
sort = {"$sort": {"total_launches": -1}}
```

```
pipeline = [match, lookup, unwind, group, project, sort]
cursor = launches_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

	total_launches	successful	failed	cost_per_launch	rocket	success_rate_pct
0	178	176	2	50000000	Falcon 9	98.9
1	5	2	3	6700000	Falcon 1	40.0
2	3	3	0	90000000	Falcon Heavy	100.0

5.2 Most active launchpads

```
# exclude upcoming launches without a result
match = {"$match": {"success": {"$in": [True, False]}}}

# group by launchpad and collect launch stats and active years
group = {"$group": {
    "_id": "$launchpad",
    "total_launches": {"$sum": 1},
    "successful": {"$sum": {"$cond": ["$success", 1, 0]}},
    "years_active": {"$addToSet": "$launch_year"}
}}

# join launchpad details from launchpads_stage by id
lookup = {"$lookup": {
    "from": "launchpads_stage",
    "localField": "_id",
    "foreignField": "_id",
    "as": "pad_info"
}}

# flatten pad_info array into a single object
unwind = {"$unwind": "$pad_info"}

# compute success rate and number of distinct years active
add_fields = {"$addFields": {
    "success_rate_pct": {"$round": [
        {"$multiply": [{"$divide": ["$successful", "$total_launches"]}, 100]},
        1
    ]},
    "years_span": {"$size": "$years_active"}
}}

# clean up output fields and rename pad name and location
project = {"$project": {
    "_id": 0,
    "launchpad": "$pad_info.full_name",
    "location": "$pad_info.location",
    "total_launches": 1,
    "successful": 1,
    "success_rate_pct": 1,
```

```
    "years_span": 1
}}

# sort by most active launchpad first
sort = {"$sort": {"total_launches": -1}}
```

```
pipeline = [match, group, lookup, unwind, add_fields, project, sort]
cursor = launches_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

	total_launches	successful	success_rate_pct	years_span	launchpad
0	99	97	98.0	12	Cape Canaveral Space Force Station Space Launc...
1	55	55	100.0	6	Kennedy Space Center Historic Launch Complex 39A
2	27	27	100.0	8	Vandenberg Space Force Base Space Launch Compl...
3	5	2	40.0	4	Kwajalein Atoll Omelek Island

5.3 Launch frequency over time

```
# exclude upcoming launches without a result
match = {"$match": {"success": {"$in": [True, False]}}}

# group by year and count total, successful and failed launches
group = {"$group": {
    "_id": "$launch_year",
    "total": {"$sum": 1},
    "successful": {"$sum": {"$cond": ["$success", 1, 0]}},
    "failed": {"$sum": {"$cond": ["$success", 0, 1]}}
}}

# compute success rate per year
add_fields = {"$addFields": {
    "success_rate_pct": {"$round": [
        {"$multiply": [{"$divide": ["$successful", "$total"]}, 100]},
        1
    ]}
}}

# sort chronologically
sort = {"$sort": {"_id": 1}}

# rename _id to year and clean up output fields
project = {"$project": {
    "_id": 0,
    "year": "$_id",
```

```
    "total": 1,
    "successful": 1,
    "failed": 1,
    "success_rate_pct": 1
}}
```

```
pipeline = [match, group, add_fields, sort, project]
cursor = launches_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

	total	successful	failed	success_rate_pct	year
0	1	0	1	0.0	2006
1	1	0	1	0.0	2007
2	2	1	1	50.0	2008
3	1	1	0	100.0	2009
4	2	2	0	100.0	2010
5	2	2	0	100.0	2012
6	3	3	0	100.0	2013
7	6	6	0	100.0	2014
8	7	6	1	85.7	2015
9	9	8	1	88.9	2016
10	18	18	0	100.0	2017
11	21	21	0	100.0	2018
12	13	13	0	100.0	2019
13	26	26	0	100.0	2020
14	31	31	0	100.0	2021
15	43	43	0	100.0	2022

5.4 Core reuse analysis

```
# unwind cores array so each core entry becomes its own document
unwind = {"$unwind": "$cores"}

# exclude entries where core id or flight number is missing
match = {"$match": {
    "cores.core": {"$ne": None},
    "cores.flight": {"$ne": None}
}}

# group by core id and collect flight and landing stats
group = {"$group": {
    "_id": "$cores.core",
    "total_flights": {"$max": "$cores.flight"},
    "landing_attempts": {"$sum": {"$cond": [
```

```
        {"$eq": ["$cores.landing_attempt", True]}, 1, 0
    ]}},
    "landing_successes": {"$sum": {"$cond": [
        {"$eq": ["$cores.landing_success", True]}, 1, 0
    ]}},
    "landing_types": {"$addToSet": "$cores.landing_type"}
}}

# bucket cores by number of total flights to see reuse distribution
bucket = {"$bucket": {
    "groupBy": "$total_flights",
    "boundaries": [1, 2, 3, 5, 8, 20],
    "default": "20+",
    "output": {
        "core_count": {"$sum": 1},
        "avg_landing_success": {"$avg": "$landing_successes"}
    }
}}

# rename _id to flight_range and round average landing successes
project = {"$project": {
    "_id": 0,
    "flight_range": "$_id",
    "core_count": 1,
    "avg_landing_success": {"$round": ["$avg_landing_success", 1]}
}}
```

```
pipeline = [unwind, match, group, bucket, project]
cursor = launches_stage.aggregate(pipeline)
pd.DataFrame(cursor)
```

	core_count	flight_range	avg_landing_success
0	45	1	0.2
1	16	2	1.6
2	6	3	2.7
3	5	5	5.8
4	6	8	11.5